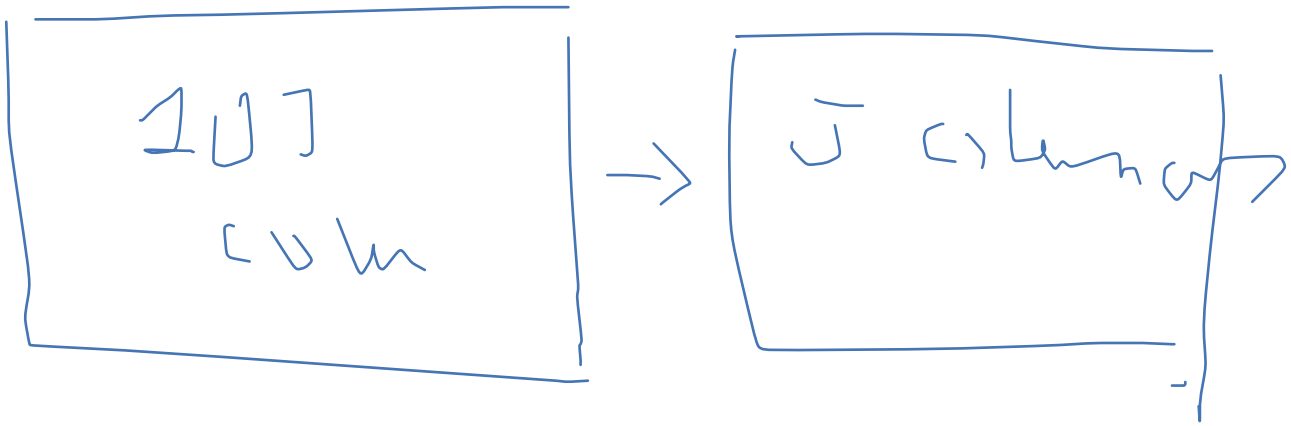
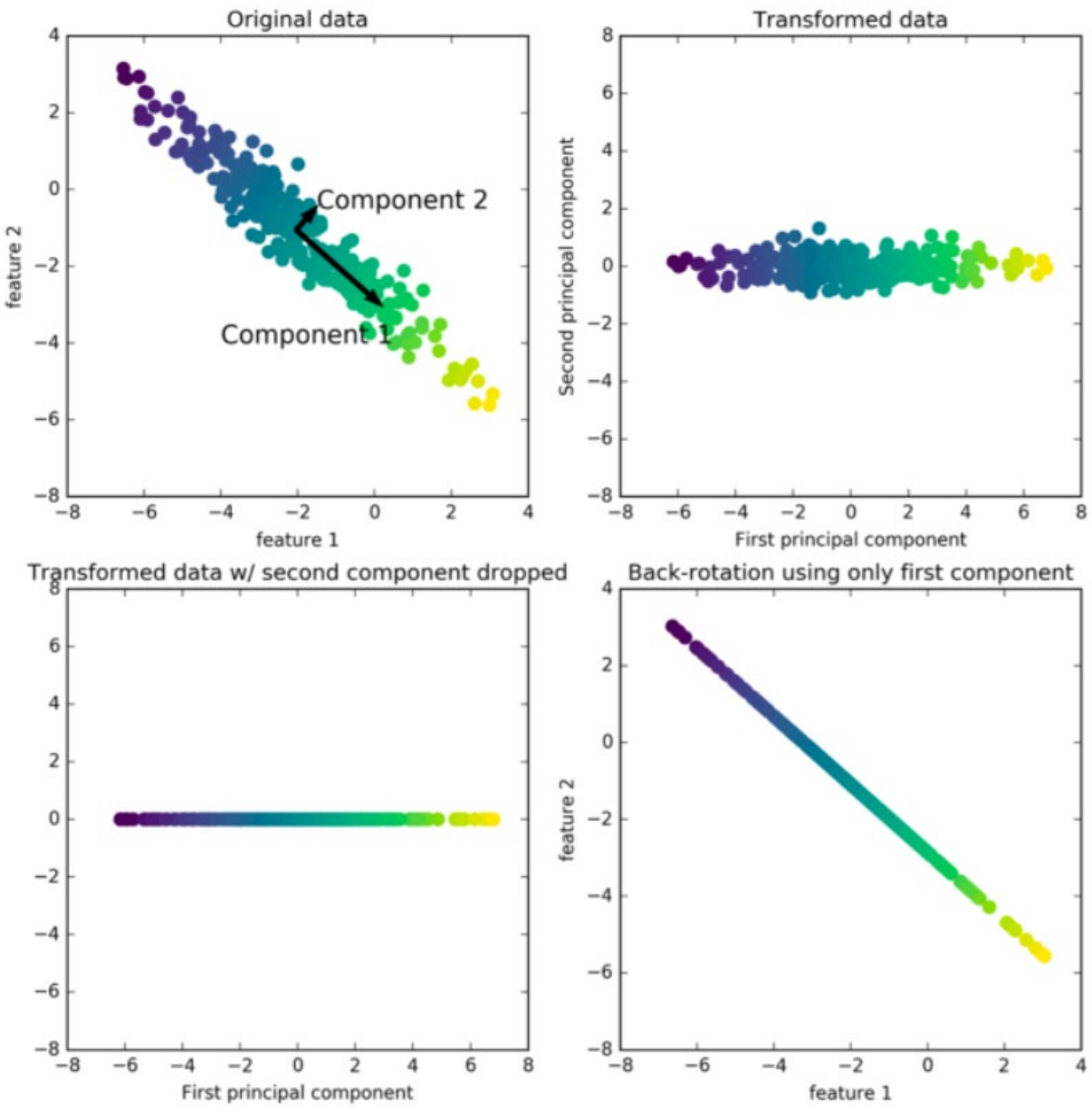
Reducción de la dimensionalidad





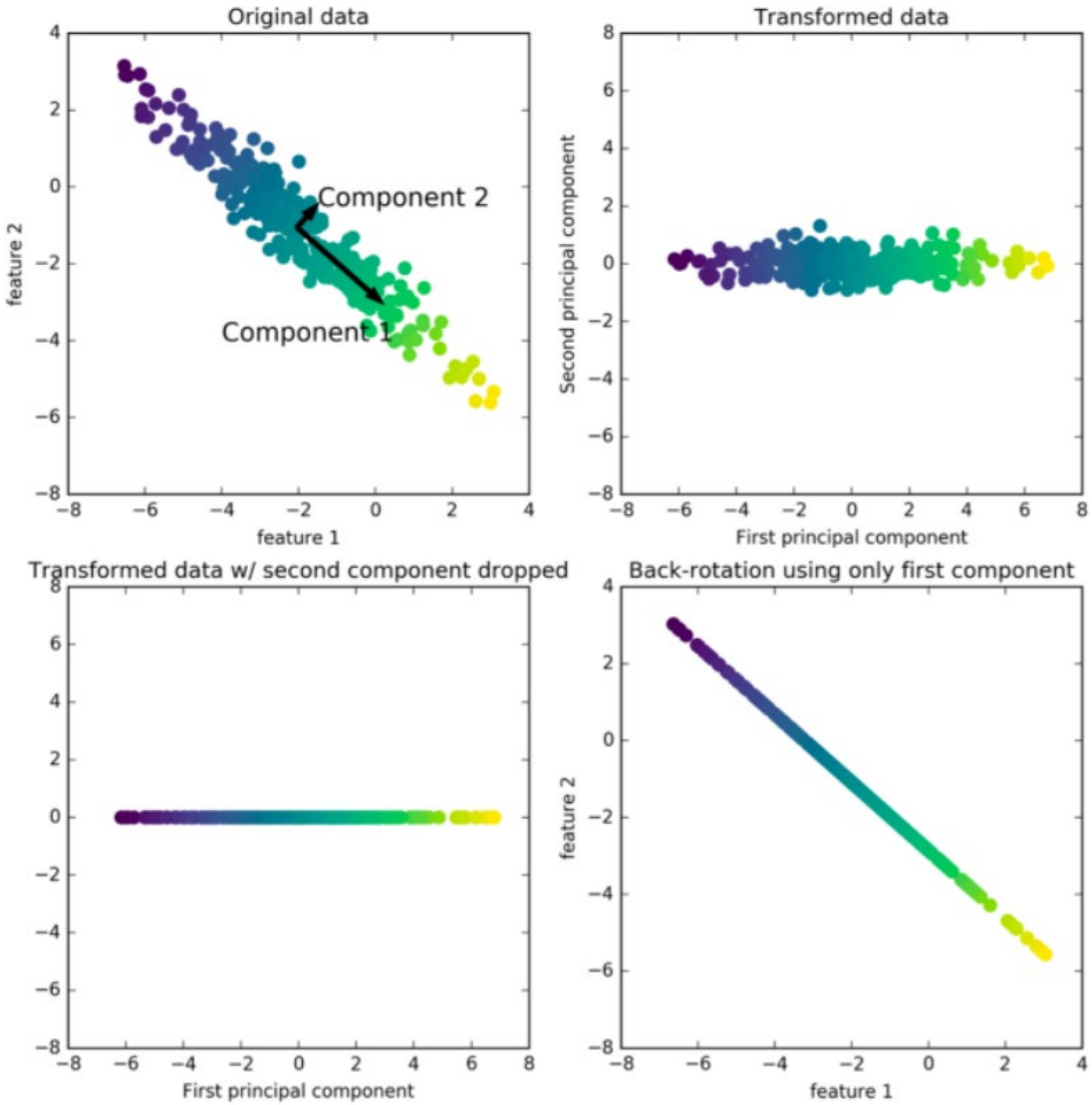
Objetivos del PCA

$$\min_{',\mathrm{rank}(X')=r} \|X - X'\|$$

Objetivos del PCA

$$\max_{u_1 \in R^p, \|u_1\|=1} \operatorname{var}(Xu_1)$$

$$\max_{u_1 \in R^p, \|u_1\|=1} u_1^T \operatorname{cov}(X) u_1$$



Cálculo del PCA

Centrar X (restar la media).
En la práctica: También escalar a la varianza unitaria.
Calcular la descomposición del valor singular:

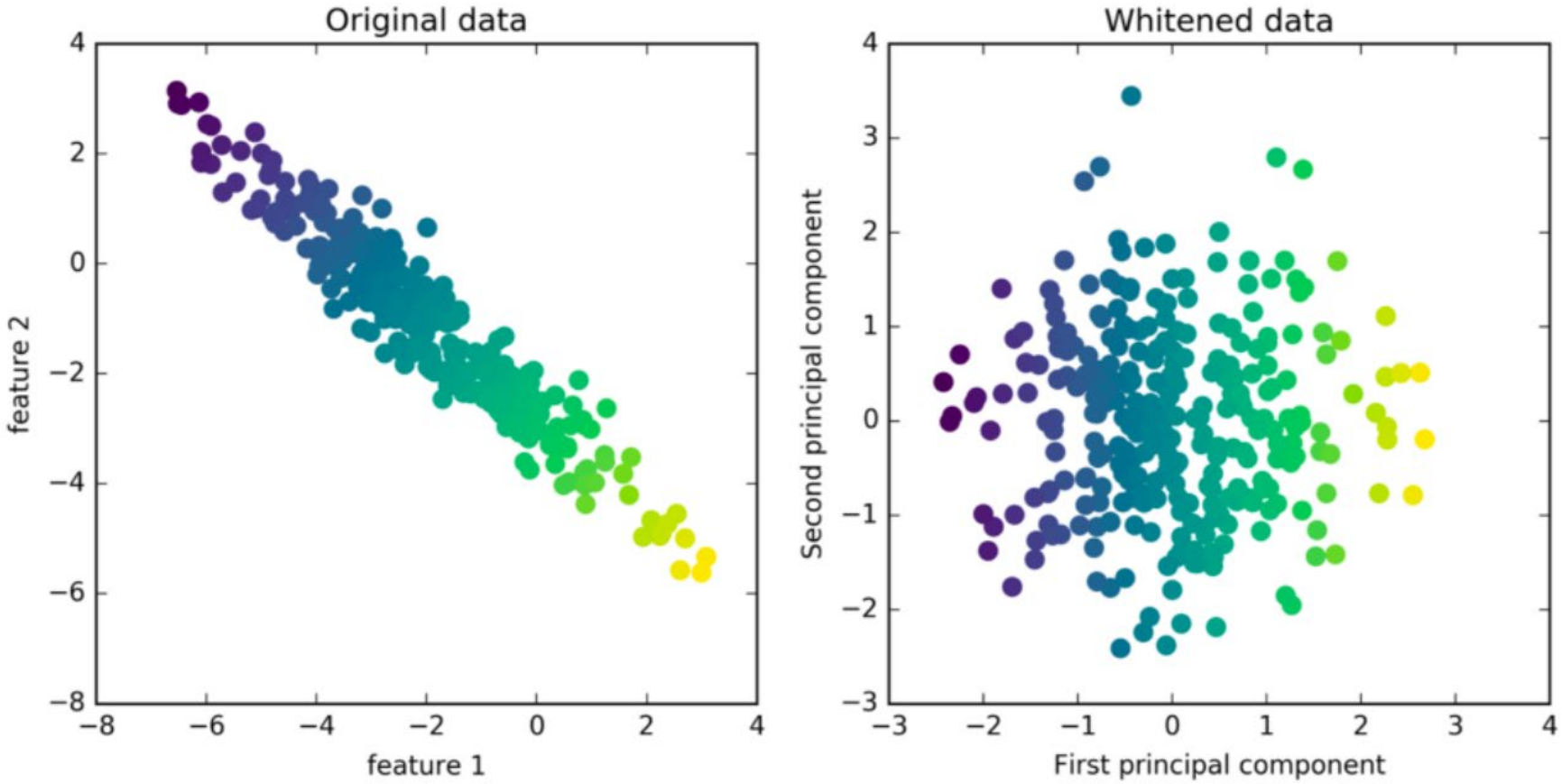
Diagonal (containing singular values)

$$X = UDV^T$$

n_samples x n_samples
orthogonal matrix
(not necessarily computed in practice)

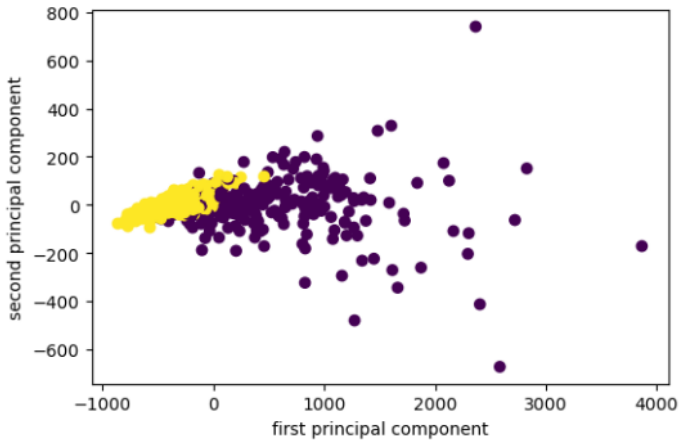
n_features x n_features
orthogonal matrix
Contains component vectors.
Drop rows (of V^T) for dimensionality reduction.

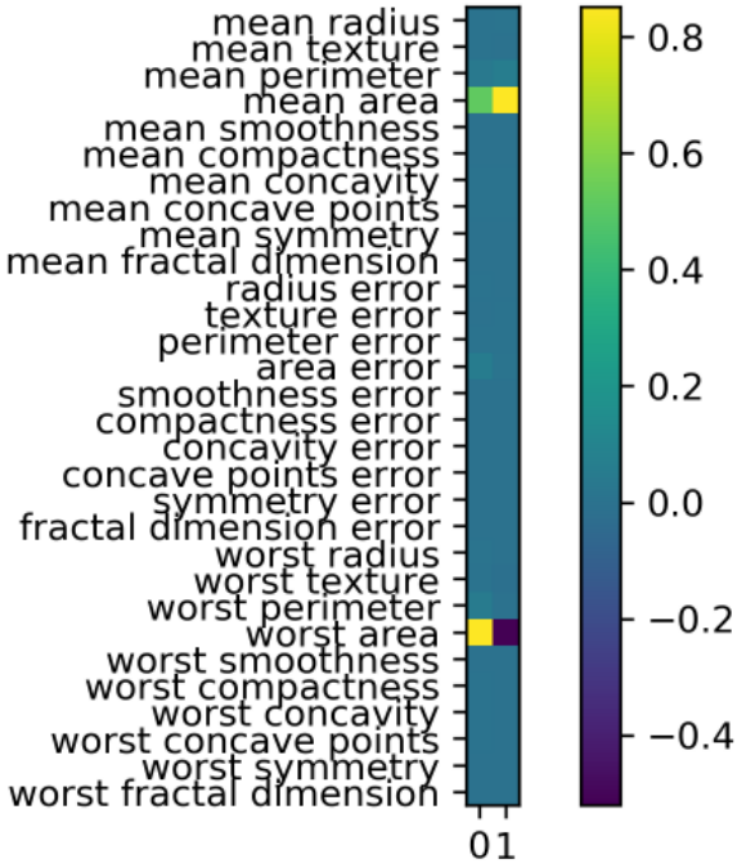
Whitening


Original data
feature 2
feature 1
4
2
0
−2
−4
−6
−8
−8
−6
−4
−2
0
2
4
Whitened data
Second principal component
First principal component
4
3
2
1
0
−1
−2
−3
−3
−2
−1
0
1
2
3
4

PCA para visualización

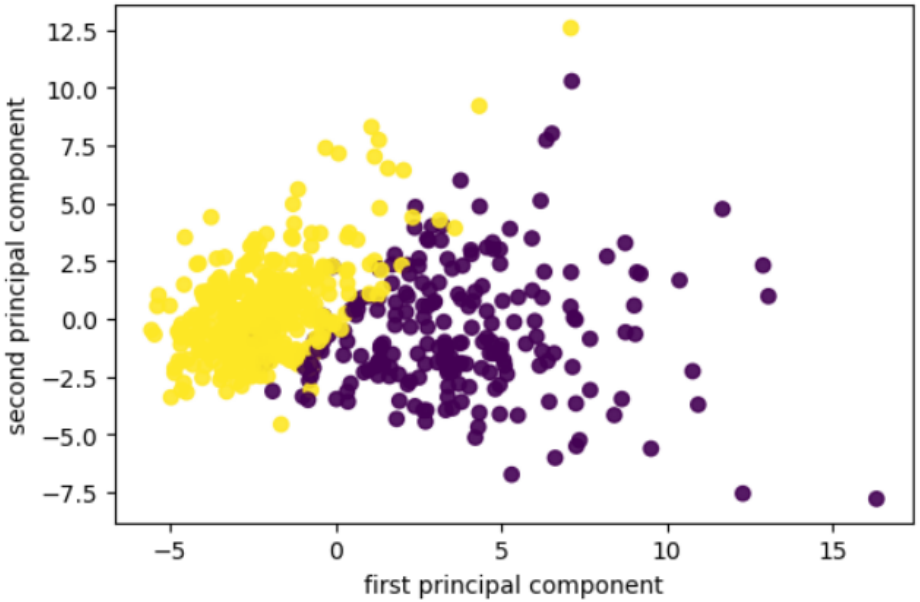
```
from sklearn.decomposition import PCA
print(cancer.data.shape)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(cancer.data)
print(X_pca.shape)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=cancer.target)
plt.xlabel("first principal component")
plt.ylabel("second principal component")
components = pca.components_
plt.imshow(components.T)
plt.yticks(range(len(cancer.feature_names)), cancer.feature_names)
plt.colorbar()
```





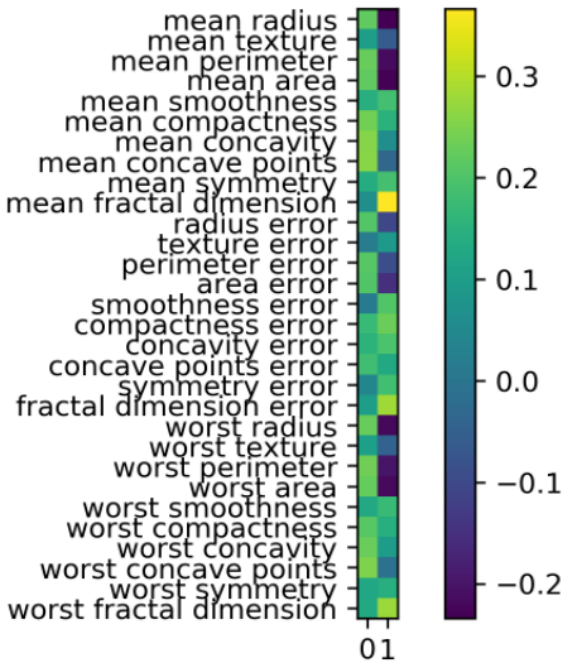
Escalamiento

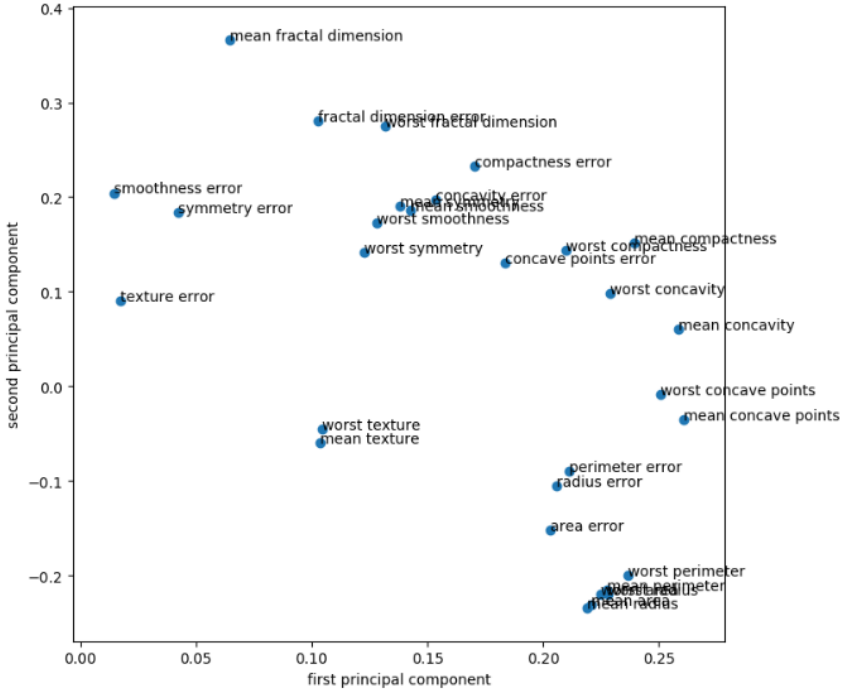
```
pca_scaled = make_pipeline(StandardScaler(), PCA(n_components=2))
X_pca_scaled = pca_scaled.fit_transform(cancer.data)
plt.scatter(X_pca_scaled[:, 0], X_pca_scaled[:, 1], c=cancer.target, alpha=.9)
plt.xlabel("first principal component")
plt.ylabel("second principal component")
```



Inspección de componentes

```
components = pca_scaled.named_steps['pca'].components_
plt.imshow(components.T)
plt.yticks(range(len(cancer.feature_names)), cancer.feature_names)
plt.colorbar()
```





PCA para regularización

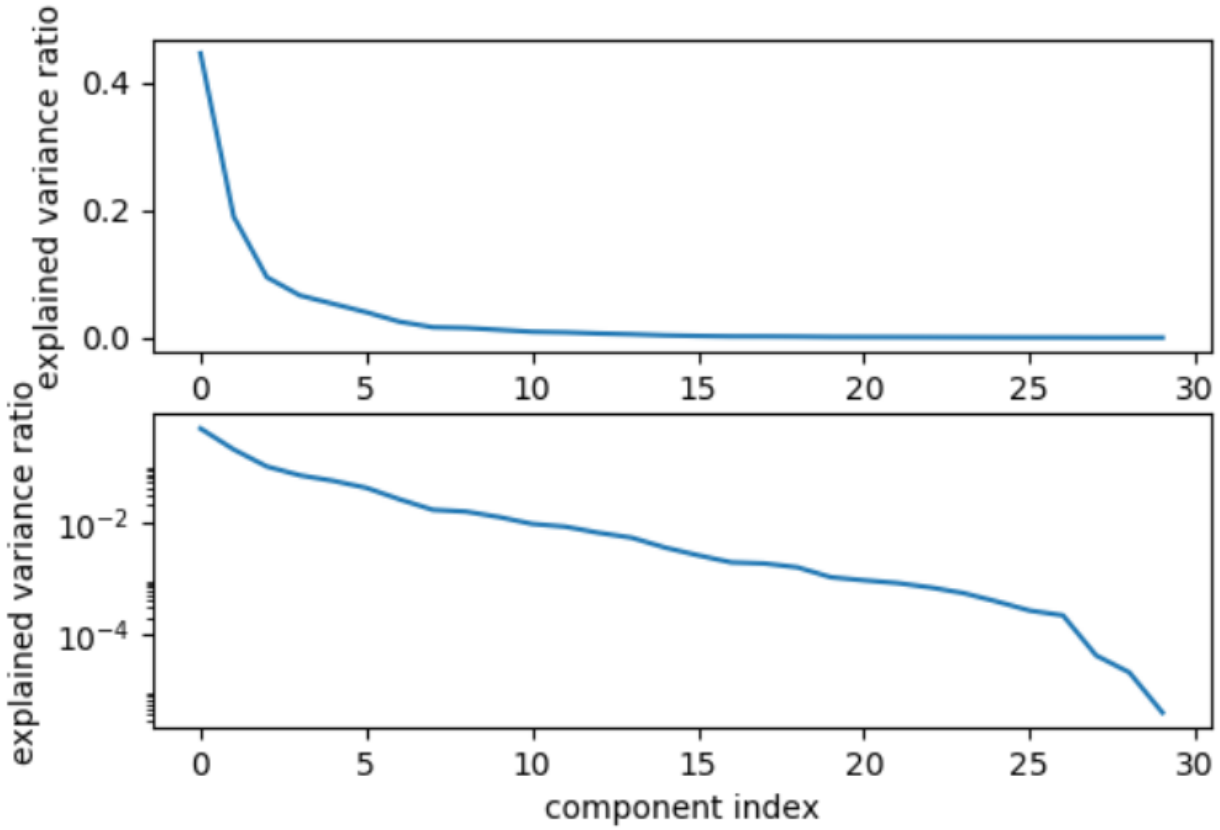
```
from sklearn.linear_model import LogisticRegression
X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target, stratify=cancer.target, random_state=0)
lr = LogisticRegression(C=10000).fit(X_train, y_train)
print(lr.score(X_train, y_train))
print(lr.score(X_test, y_test))
```

```
0.993
0.944
```

```
pca_lr = make_pipeline(StandardScaler(), PCA(n_components=2), LogisticRegression(C=10000))
pca_lr.fit(X_train, y_train)
print(pca_lr.score(X_train, y_train))
print(pca_lr.score(X_test, y_test))
```

```
0.961
0.923
```

Cobertura de la varianza



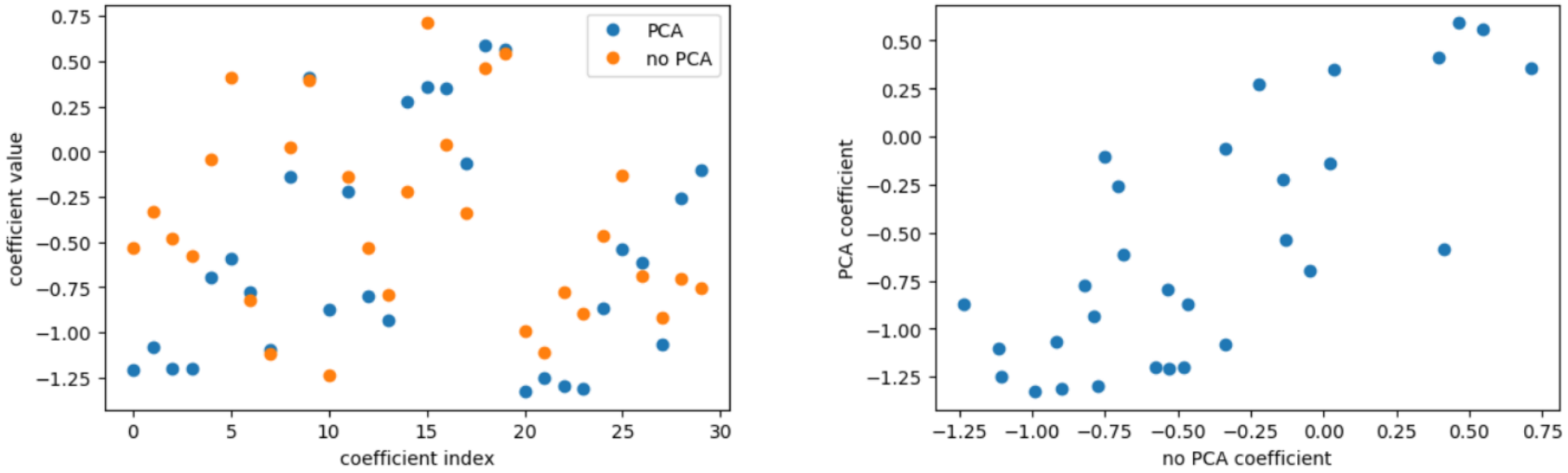

```
pca_lr = make_pipeline(StandardScaler(), PCA(n_components=6), LogisticRegression(C=10000))
pca_lr.fit(X_train, y_train)
print(pca_lr.score(X_train, y_train))
print(pca_lr.score(X_test, y_test))
```

```
0.981
0.958
```

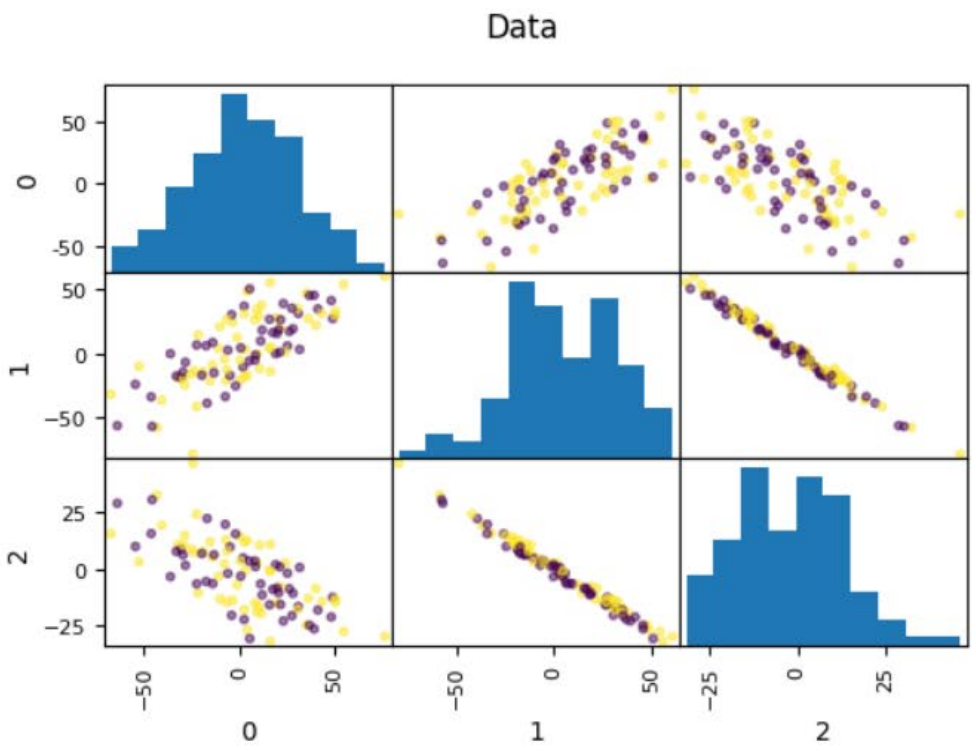
Interpretación de los coeficientes

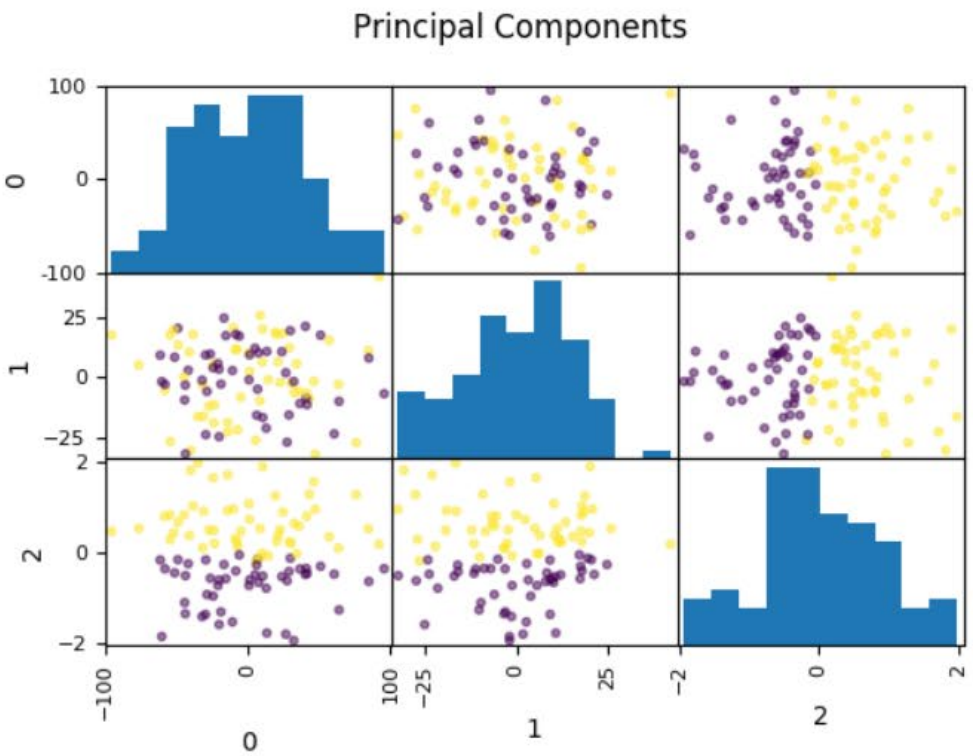
```
pca = pca_lr.named_steps['pca']
lr = pca_lr.named_steps['logisticregression']
coef_pca = pca.inverse_transform(lr.coef_)
```

Comparing PCA + Logreg vs plain Logreg:

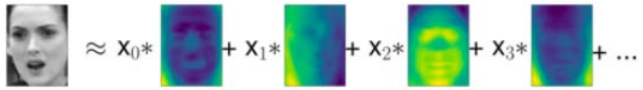


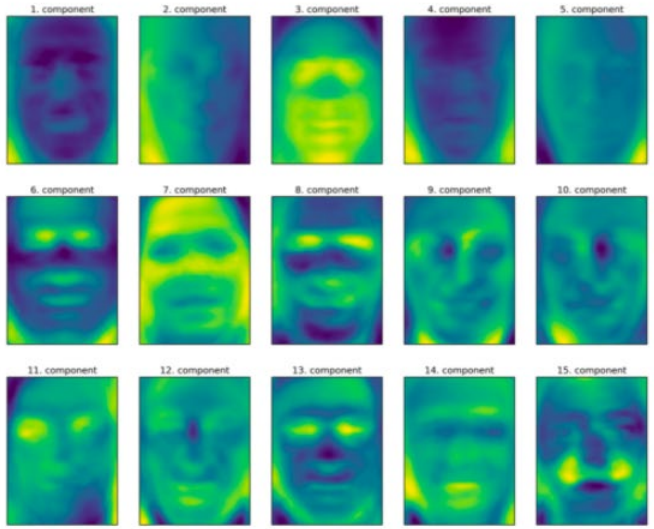
PCA es non supervisado





PCA para extracción de características





Remember:
Signs don't mean anything!

1-NN and Eigenfaces

```
from sklearn.neighbors import KNeighborsClassifier
# split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_people, y_people, stratify=y_people, random_state=0)
print(X_train.shape)
# build a KNeighborsClassifier using one neighbor
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(X_train, y_train)
knn.score(X_test, y_test)
```

```
(1547, 5655)
0.23
```

```
pca = PCA(n_components=100, whiten=True, random_state=0).fit(X_train)
X_train_pca = pca.transform(X_train)
X_test_pca = pca.transform(X_test)
X_train_pca.shape
```

```
(1547, 100)
```

```
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(X_train_pca, y_train)
knn.score(X_test_pca, y_test))
```

```
0.31
```

Reconstrucción

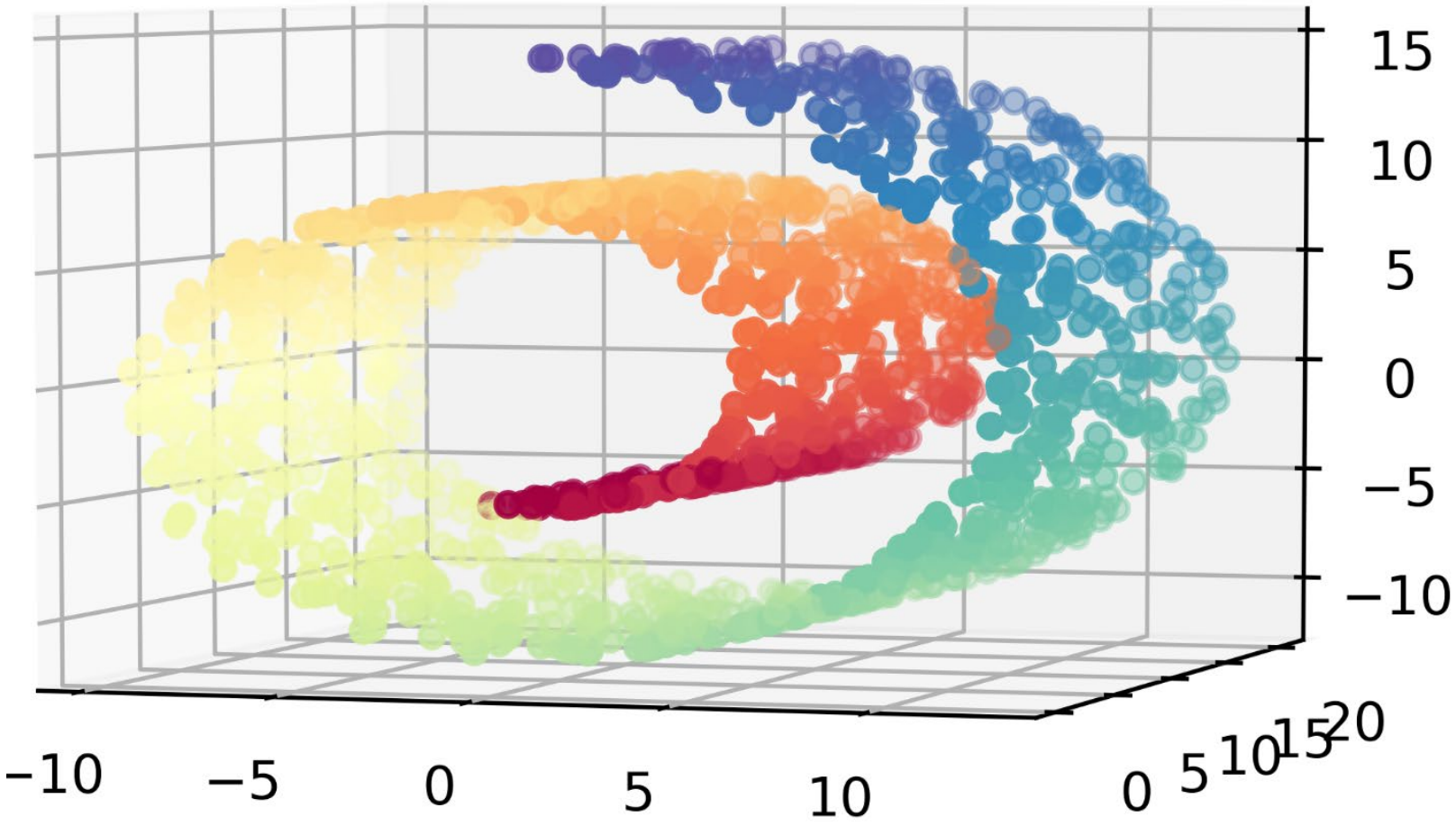
PCA para detección de outliers

```
pca = PCA(n_components=100).fit(X_train)
reconstruction_errors = np.sum((X_test - pca.inverse_transform(pca.transform(X_test))) ** 2, axis=1)
```

Best reconstructions

Worst reconstructions

Manifold Learning



- KernelPCA - hace PCA, ¡pero con kernels!
- Valores propios de la matriz del núcleo
- Incrustación espectral (mapas propios del laplaciano)
- Utiliza los valores propios del laplaciano del gráfico
- Incrustación localmente lineal
- Isomap "kernel PCA on manifold" (núcleo PCA en un colector)
- t-SNE (incrustación estocástica de vecinos distribuida en t)

t-SNE

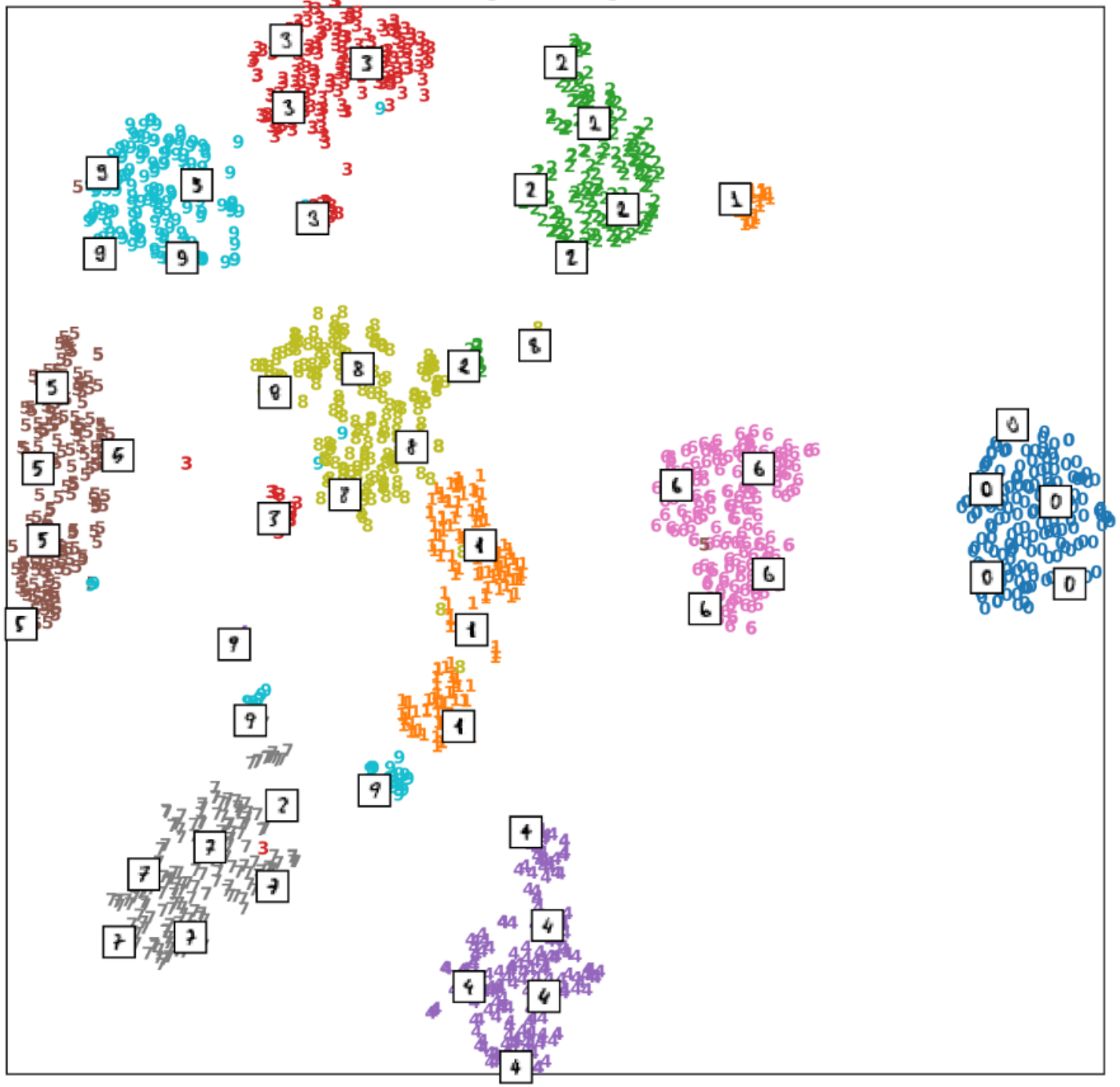
$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2/2\sigma_i^2)}{\sum_{k\neq i}\exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2/2\sigma_i^2)}$$

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

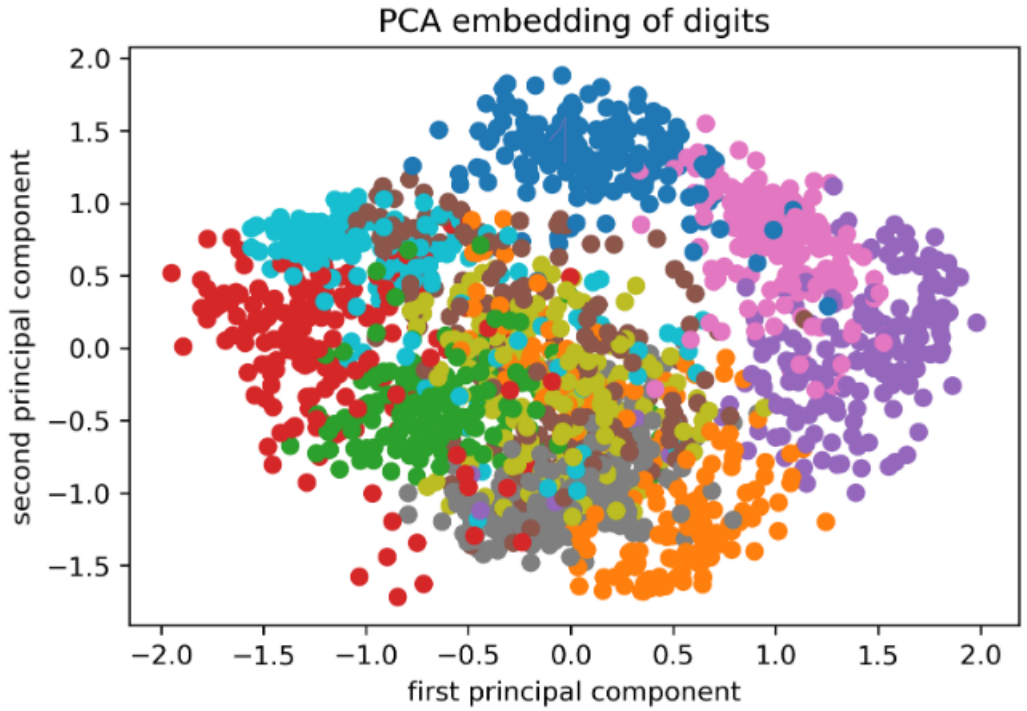
$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k\neq i}(1 + \|\mathbf{y}_i - \mathbf{y}_k\|^2)^{-1}}$$

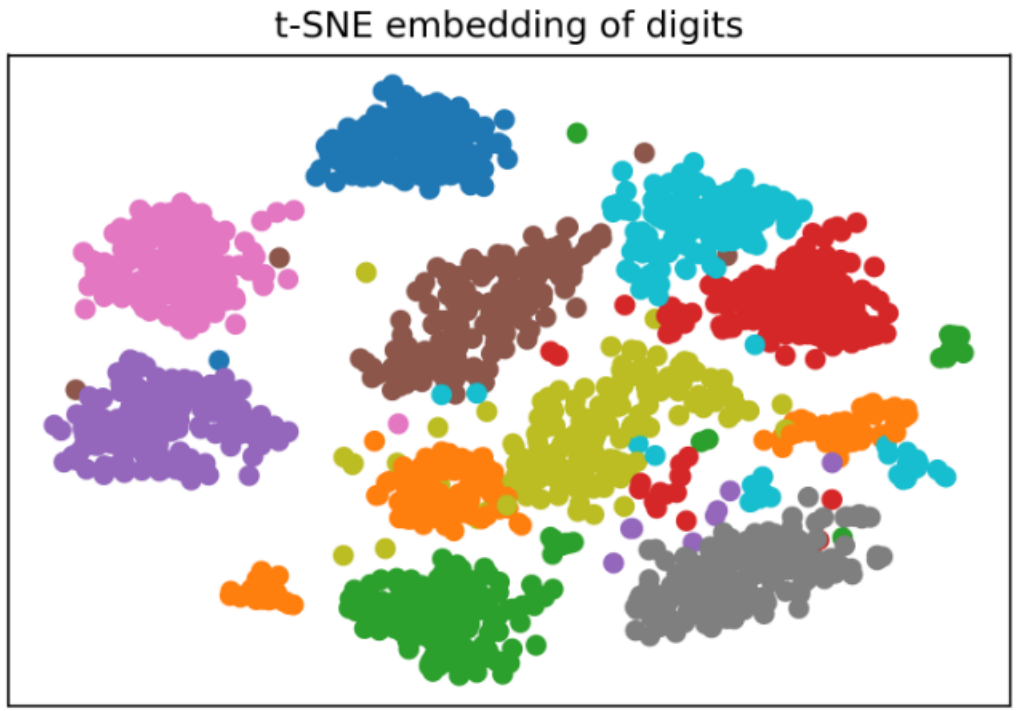
$$KL(P||Q) = \sum_{i\neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

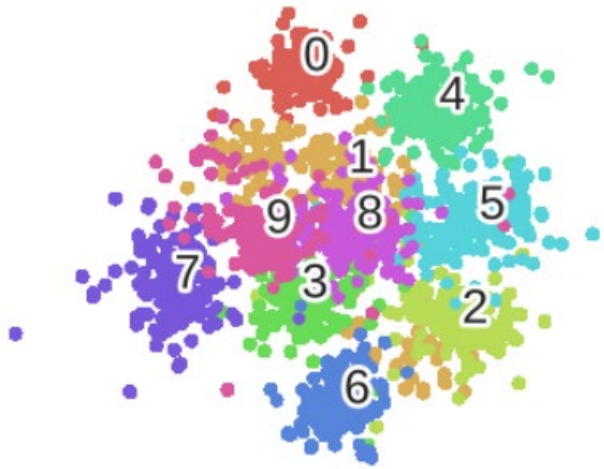
t-SNE embedding of the digits (time 11.13s)

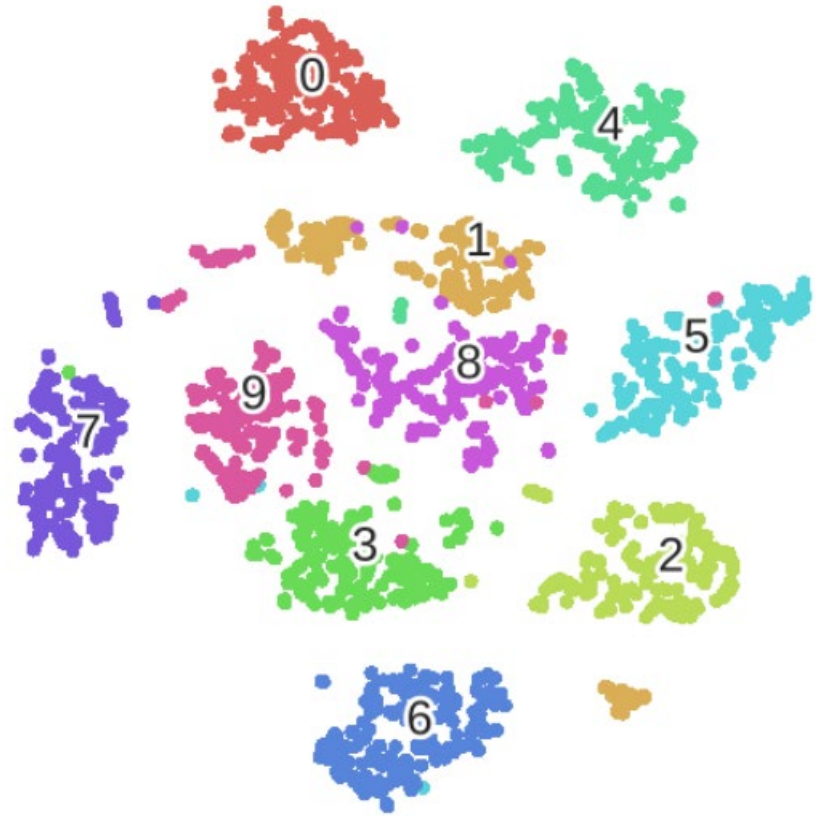


```python
from sklearn.manifold import TSNE
from sklearn.datasets import load_digits
digits = load_digits()
X = digits.data / 16.
X_tsne = TSNE().fit_transform(X)
X_pca = PCA(n_components=2).fit_transform(X)
```

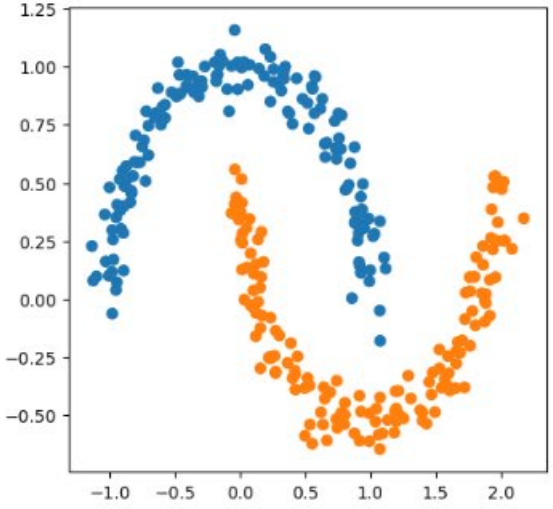


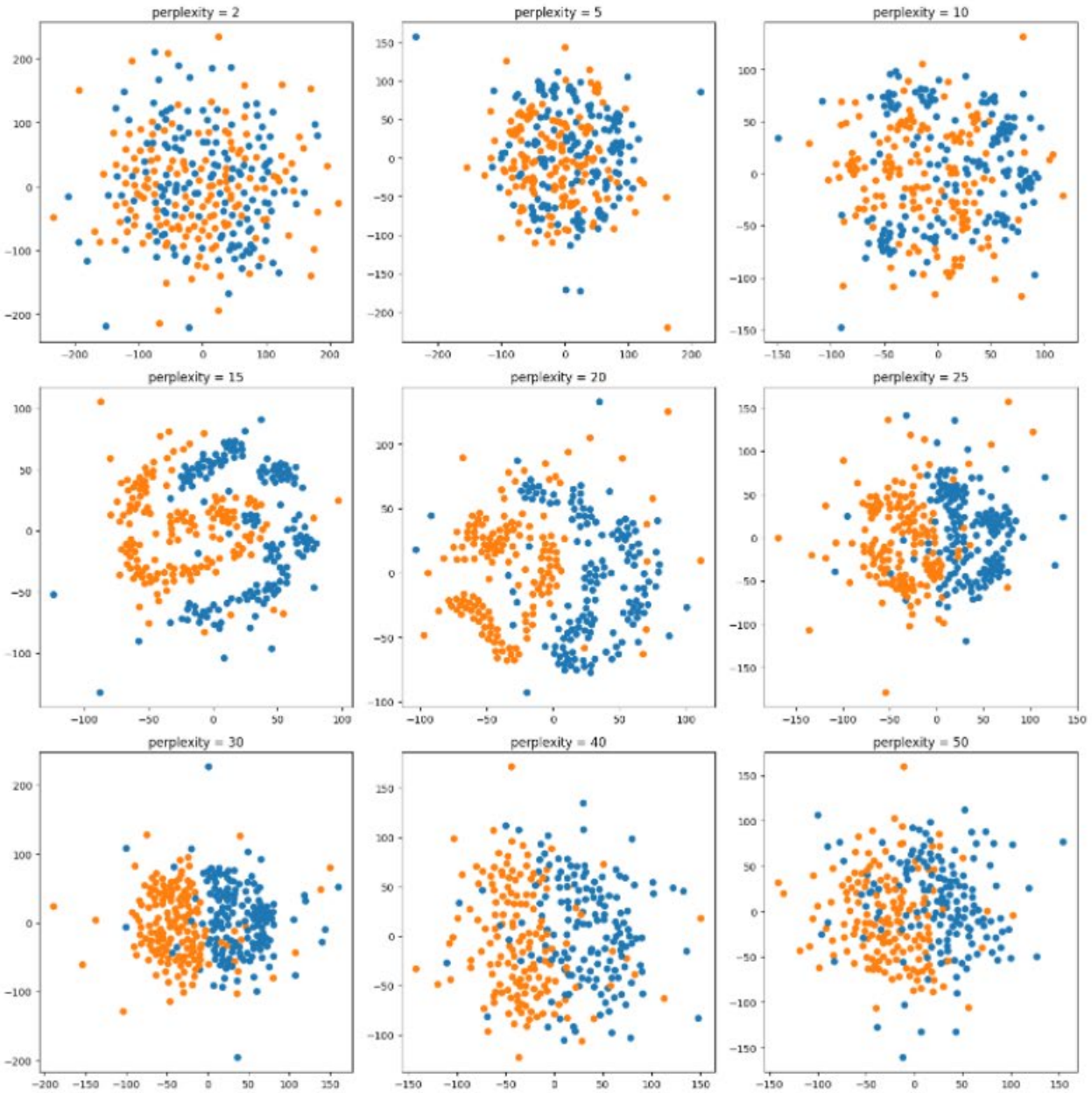






[GitHub - oreillymedia/t-SNE-tutorial: A tutorial on the t-SNE learning algorithm]

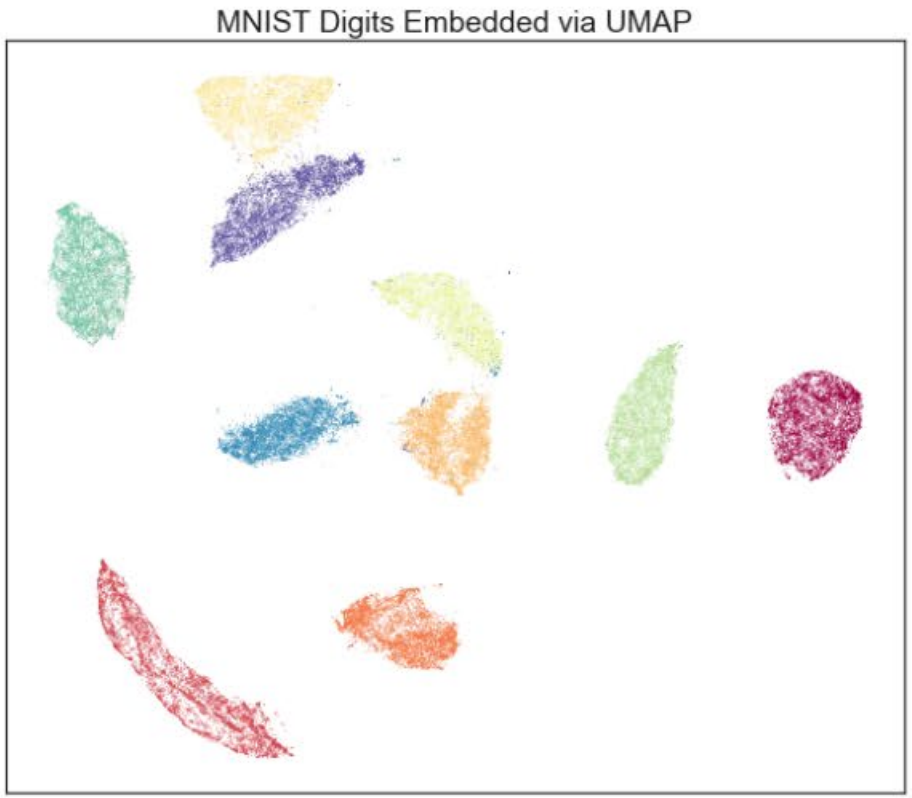


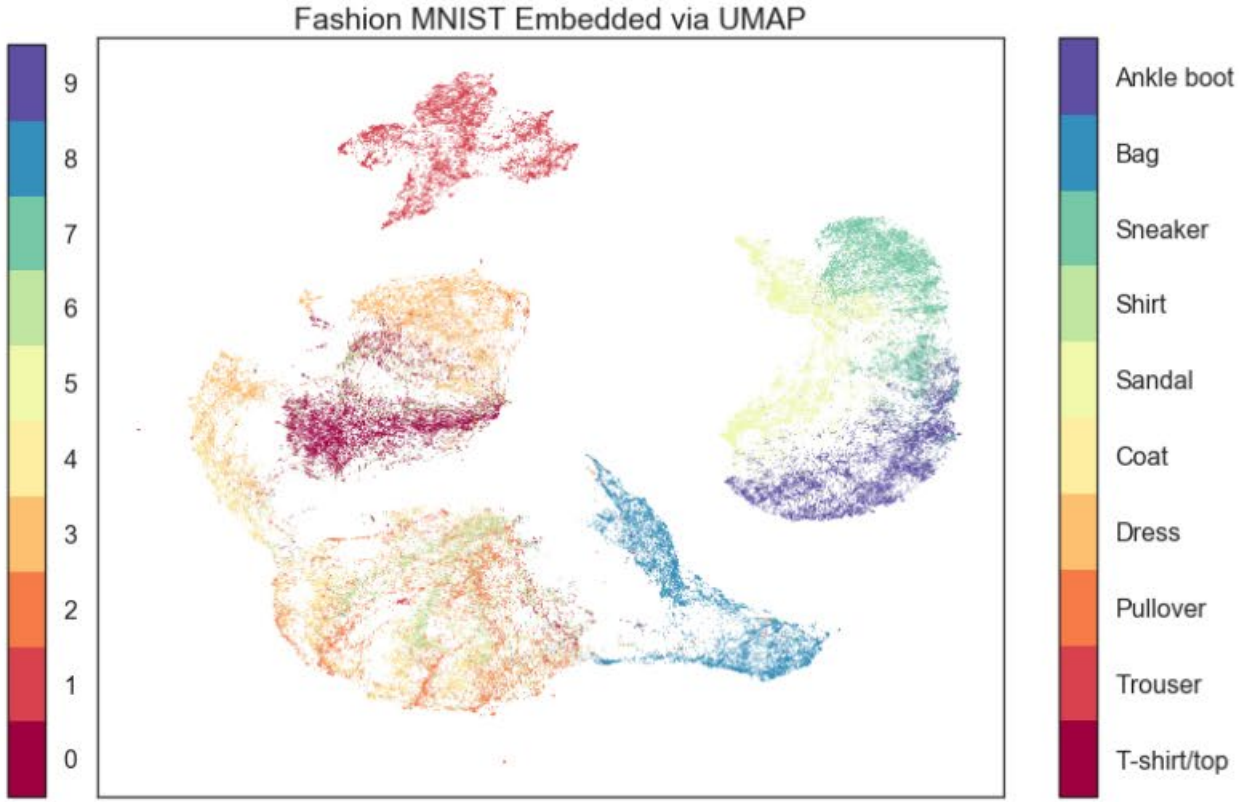

perplexity = 2
perplexity = 5
perplexity = 10
perplexity = 15
perplexity = 20
perplexity = 25
perplexity = 30
perplexity = 40
perplexity = 50

Uniform Manifold Approximation and Projection (UMAP)

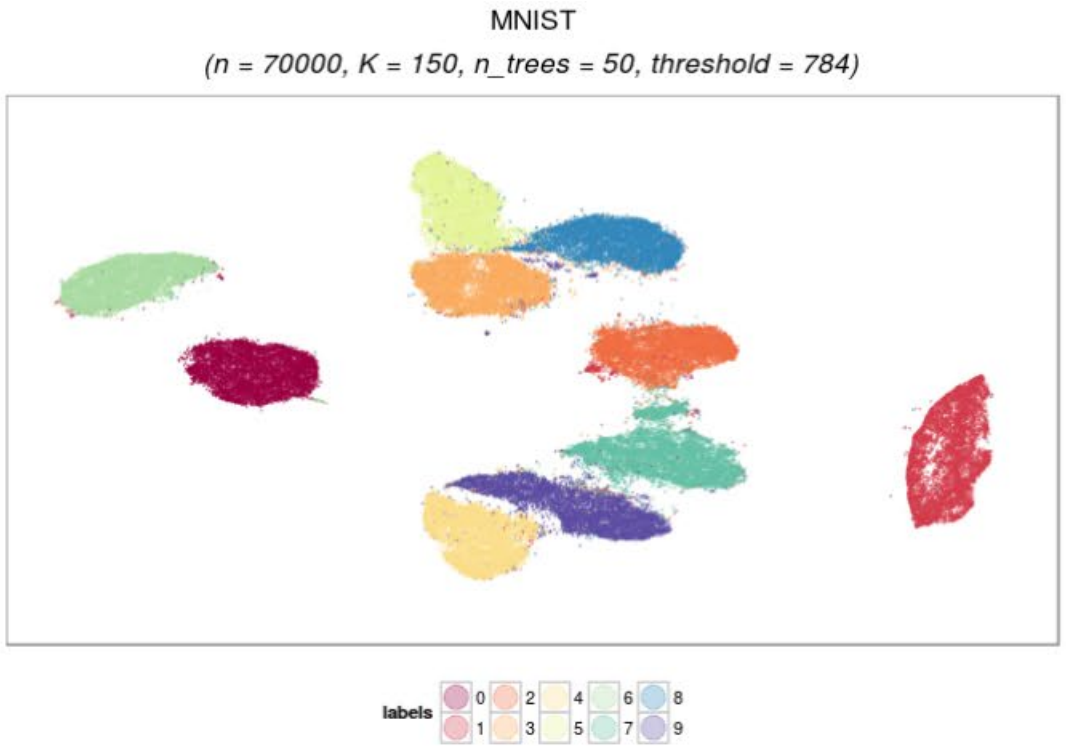
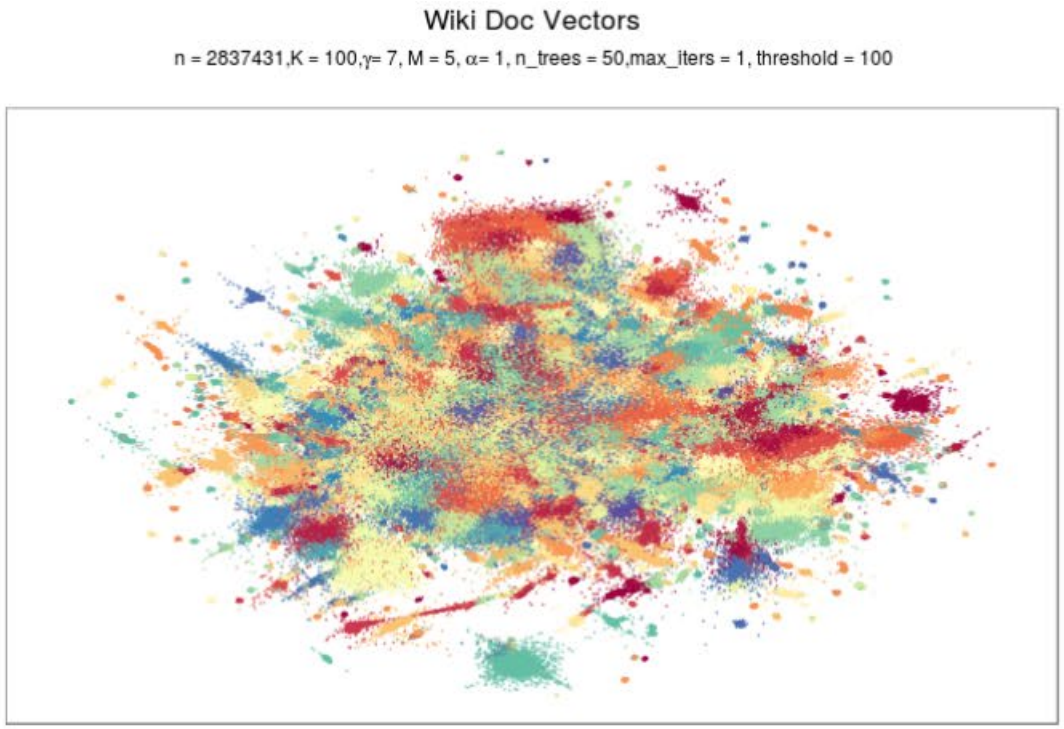
- https://github.com/lmcinnes/umap

- Construye el gráfico a partir de símiles, utiliza el algoritmo de disposición del gráfico
- Mucho más rápido que t-SNE: MNIST tarda 2 minutos frente a 45 minutos de t-SNE





largeVis

- https://github.com/elbamos/largeVis

- Utiliza el gráfico del vecino más cercano construido de forma muy eficiente (y aproximada)
- Mucho más rápido que t-SNE